DH2323 DGI

Lab 2: Raytracing

Processing (Java)

In this lab you will implement a Raytracer, which draws images of 3D scenes by tracing the light rays reaching the simulated camera. The lab is divided into several steps. To get something on the screen, you first need to learn how to:

- Represent 3D scenes using triangular surfaces.
- Trace the ray of each pixel in the camera image into the scene.
- Compute ray-triangle intersections, to find out which surface a ray hit.

You will then be able to write a program that draws the image seen to the left in Figure 1. Next, you will extend this program by adding:

- Camera motion.
- Simulation of light sources.
- Reflection of light for diffuse surfaces

The result of this can be seen in **Figure 1**. To start, you should create a new directory for your Processing sketch and then place the downloaded **DGILab2.pde** and **utils.pde** source files in to it.

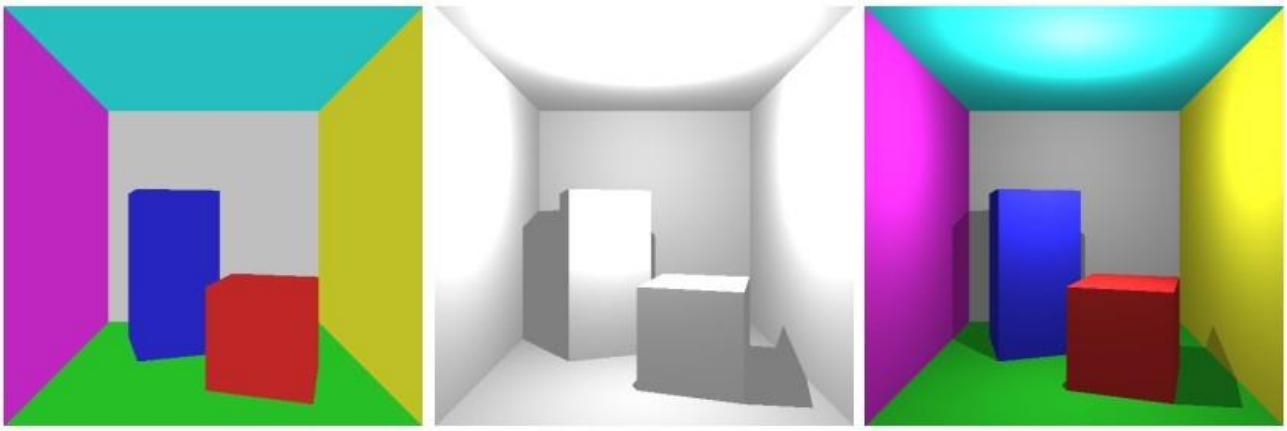

Figure 1: The output from this lab.

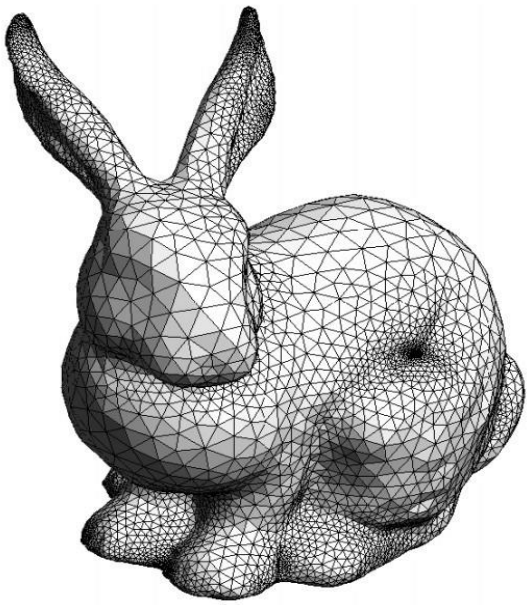

Figure 2: A 3D model represented by triangular surfaces. This bunny is a common test model in computer graphics. It can be found at: **http://graphics.stanford.edu/data/3Dscanrep/**

1 Representing Surfaces

In the first lab you coded a star-field by simulating a pinhole camera and its projection of moving 3D points. In this lab you will learn how to represent and draw surfaces, instead of points. The simplest surface to describe mathematically is a triangular surface. It is simple because it is planar and its geometrical properties can be described by only three 3D points: its vertices. For instance, from the three vertices we can compute the normal of the surface, i.e. a vector pointing out from the surface, being orthogonal to the surface itself. Although one can use other representations for surfaces triangles are the most used one for computer graphics due to its simplicity.

See Figure 2 for an example of a 3D model represented by triangular surfaces. When describing a surface for computer graphics it is not only the geometrical properties of the surface that are of interest. We are also interested in its appearance. The simplest way to describe the appearance of a triangle is with a single color. We will therefore use the following data structure to represent a triangle (see the *Triangle* class):

```
class Triangle {
      public PVector v0, v1, v2;
      public PVector normal;
      public PVector col;
      …
}
```

See the *Processing programming reference*[1] for details about *PVector*. Although the normal can be computed from the vertices, it can be convenient to have it in the data structure as well. Then the

[1] https://processing.org/reference/

normals can be computed (see the *computeNormal();* function) and stored so that we can easily access them later whenever we want, without any computations.

We can now represent a 3D model by a set of triangles. To store all these triangles we will use a global ArrayList as follows:

```
static ArrayList<Triangle> triangles;
```

More information about *ArrayLists*, part of the *java.util* package, can be found here: https://www.w3schools.com/java/java_arraylist.asp

To fill this vector with triangles representing a 3D model you can use the function:

```
loadTestModel(triangles);
```

The function and the Triangle-struct and other useful functions are defined in the file **utils.pde** which you should make sure to include in your Processing sketchbook. This function will fill the vector it takes as an argument with the triangles describing the model seen in **Figure 1**. This is a famous test model in computer graphics. You can read more about it at **http://www.graphics.cornell.edu/online/box/**. It is a cubic room with dimensions:

$$-1 \leq x \leq 1 \qquad (1)$$

$$-1 \leq y \leq 1 \qquad (2)$$

$$-1 \leq z \leq 1 \qquad (3)$$

2 Intersection of Ray and Triangle

In our raytracer we want to trace rays, for each pixel of the image, to see where they come from. Since we use triangular surfaces to represent our model, this corresponds to finding the first intersection between a triangle and a ray. We will now derive equations for this computation.

2.1 Triangles in 3D

Assume v_0, v_1, v_2 are real 3D vectors representing the vertices of the triangle, i.e. they are elements of $\mathbb{R}^3$. To describe a point in the triangle we construct a coordinate system that is aligned with the triangle. We let v_0 be the origin of the coordinate system and we let the axis be parallel to two of the edges of the triangle:

$$e_1 = v_1 - v_0 \qquad (4)$$

$$e_2 = v_2 - v_0 \qquad (5)$$

$e_1 \in \mathbb{R}^3$ is then a vector that is parallel to the edge of the triangle between v_0 and v_1. Similarly, $e_2 \in \mathbb{R}^3$ is parallel to the edge between v_0 and v_2. Any point $r \in \mathbb{R}^3$ in the plane of the triangle can then be described by two scalar coordinates $u \in \mathbb{R}$ and $v \in \mathbb{R}$, such that its position in 3D is:

$$r = v_0 + ue_1 + ve_2 \qquad (6)$$

This holds for all points r that are in the plane of the triangle. For points that are not only in the same plane, but actually within the triangle we also have that:

$$0 < u \qquad (7)$$

$$0 < v \qquad (8)$$

$$u+v < 1 \qquad (9)$$

2.2 Rays in 3D

Next, we need a way to describe points that are on a ray. By a ray we mean a segment of a line in 3D. We define a 3D line by a vector representing its start position $s \in \mathbb{R}^3$ and a vector representing its direction $d \in \mathbb{R}^3$. All points r lying on the line can then be written:

$$r = s + td \qquad (10)$$

where $t \in \mathbb{R}$ is a scalar coordinate describing the position on the line. It is the signed distance, in units of $||d||$, to the position r from the start position s. If we do not put any constrain on t we are describing a line that extends infinitely in both directions. However, by a ray we only mean the points that comes after the start position. For the ray we thus require:

$$0 \leq t \qquad (11)$$

2.3 Intersection

To get the intersection between the plane of the triangle and the line of the ray we can insert the equation of the plane (6) into the equation of the line (10) and solve for the coordinates t, u, v:

$$v_0 + ue_1 + ve_2 = s + td \qquad (12)$$

This is an equation of 3D vectors. We thus have one equation for each vector component, i.e. three. And since we have three unknowns (t, u, v) it can be solved. To find the solution we re-formulate the equation using matrices, as one typically does when solving linear systems of equations:

$$v_0 + ue_1 + ve_2 = s + td \qquad (13)$$

$$td + ue_1 + ve_2 = s - v_0 \qquad (14)$$

$$(-d \quad e_1 \quad e_2) \quad (t \quad u \quad v)^T = s - v_0 \qquad (15)$$

This might still look a bit weird. To express the linear equation on the standard form we introduce the 3x3 matrix:

$$A = (-d \quad e_1 \quad e_2) \qquad (16)$$

where the vectors -d, e_1, e_2 are the columns of the matrix. We also give a new name for our right-hand side vector:

$$b = s - v_0 \qquad (17)$$

And we put our unknowns in the vector:

$$x = (t \quad u \quad v)^T \qquad (18)$$

The linear **Equation 15** can then be written more concisely as:

$$Ax = b \tag{19}$$

And the solution x can be found by multiplying both sides with the inverse of the matrix A:

$$Ax = b \tag{20}$$
$$A^{-1}Ax = A^{-1}b \tag{21}$$
$$Ix = A^{-1}b \tag{22}$$
$$x = A^{-1}b \tag{23}$$

Thus, if we can compute the inverse of the matrix A we can easily and the intersection point x. The **utils.pde** file both has a class to represent 3x3 matrices and a function to compute matrix inverses. We can therefore do the intersection computations with code that looks similar to this:

```
PVector v0 = triangle.v0;

PVector v1 = triangle.v1;

PVector v2 = triangle.v2;

PVector e1 = v1;

e1.sub(v0);

PVector e2 = v2;

e2.sub(v0);

PVector b = start;

start.sub(v0);

//TODO: initialise matrix values vals based on dir, e1 and e2

Matrix3x3 A = new Matrix3x3(vals);

Matrix3x3 Ainv = A.inverse();

PVector x = Ainv.multiply(b);
```

Now we know the coordinates t,u,v for the intersection point. We can then check the inequalities **7, 8, 9, 11** to see whether the intersection occurred within the triangle and after the beginning of the ray. The function containing the code above can look like this:

```
public static boolean ClosestIntersection(
      PVector start,
      PVector dir,
      ArrayList<Triangle> triangles,
      Intersection closestIntersection)
{
      //your code here…
}
```

It takes the start position of the ray and its direction and the ArrayList of triangles as input. It should then check for intersection against all triangles in the model. Example, the code to iterate over all triangles in a model may look like this:

```
for (int i = 0; i < triangles.size(); i++) {

      Triangle triangle = triangles.get(i);

      //your code here…

}
```

If an intersection occurred it should return true. Otherwise false. In the case of an intersection it should also return some information about the closest intersection. Define the following data structure to store this information:

```
public static class Intersection {

    public PVector position;

    public float distance;

    public int triangleIndex;

    public Intersection() {

        position = new PVector();

        distance = Float.MAX_VALUE;

        triangleIndex = -1;

}
```

The argument closestIntersection sent to the function should be updated with the 3D position of the closest intersection. Its distance from the start of the ray and the index of the triangle that was intersected.

If you are into linear algebra a good optional exercise is to calculate a closed form solution for the inverse of the matrix, instead of just using the matrix inverse() function provided in **utils.pde**. This can be done with Cramer's rule. We need this result if we would like to increase the speed of the intersection computations, which will be the bottleneck of our program. Then, when we have the closed form solution we can postpone the computation of some of the matrix elements, until they are needed. We start by just computing the row needed to compute the distance t. Then, if t is negative there is no need to continue analyzing that intersection, as it will not occur. However, doing this is not necessary to pass the lab.

3 Tracing Rays

Now that you have a function that takes a ray and finds the closest intersecting geometry you have all you need to start rendering an image with raytracing. The idea is that you trace a ray for each pixel of the image. You then need to compute for each pixel what ray it corresponds to. Initially we assume that the camera is aligned with the coordinate system of the model with x-axis pointing to the right and y-axis pointing down and z-axis pointing forward into the image. The start position for all rays corresponding to pixels is the camera position. A vector $d \in \mathbb{R}^3$ pointing in the direction where the light reaching pixel (x,y) comes from can then be computed as:

$$d = (x-W/2,\ y-H/2,\ f) \qquad (24)$$

where W and H are the width and height respectively of the image and f is the focal length of the camera, measured in units of pixels. We strongly encourage you to draw a Figure of the pinhole camera and make sure that you understand this equation. The focal length can be thought of as the distance from the hole in the pinhole camera to the image plane. Add the following global variables to store the camera parameters:

```
static float focalLength = …;

static PVector cameraPosition = new PVector (…,…,…);
```

Fill in ... with good values for these variables. Make sure that the model can be seen from the camera position you choose. What will the horizontal and vertical field of view be for the focal length you have chosen?

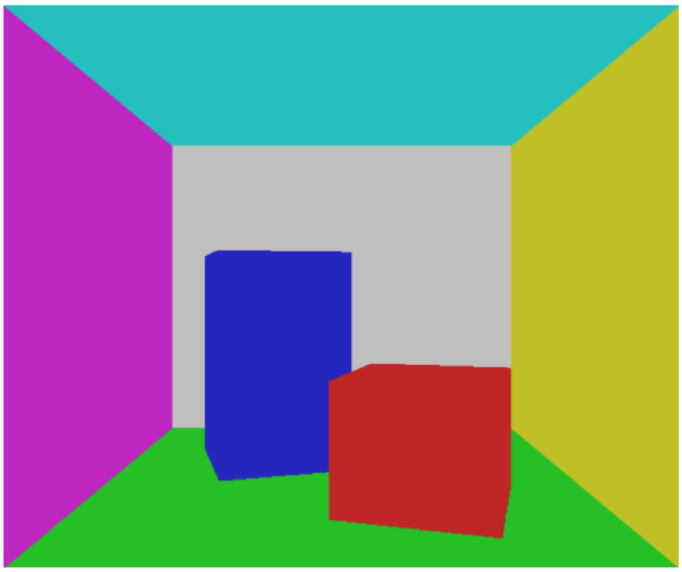

Figure 3: Tracing a light ray for each pixel to see which surface it first intersects.

You are now ready to write the Draw function. In it you should loop through all pixels and compute the corresponding ray direction. Then call the function ClosestIntersection to get the closest intersection in that direction. If there was an intersection the color of the pixel should be set to the color of that triangle. Otherwise it should be black. The result should be similar to **Figure 3**.

4 Moving the Camera

We now have a simple visualization of our 3D scene. However, it would be nice if we could move the camera around. First try to implement translation of the camera by updating the global variable cameraPos if the user presses the arrow keys. You can use Processing to check if a key is pressed in this way:

```
void keyPressed() {

  switch(key)

  {

    case 27:  //ESCAPE key

      exit();

      break;

    case CODED:

      if (keyCode == UP) {

        // Move camera forward
```

```
      } else if (keyCode == DOWN) {
        // Move camera backwards
      } else if (keyCode == LEFT) {
        // Move camera to the left
      } else if (keyCode == RIGHT) {
        // Move camera to the right
      }
      break;
    case 'w':
        System.out.println("Pressed w");
      break;
  }
}
```

Add code that translates the camera. Since raytracing is slow you will need to set a low width and height of the rendered image to get real-time performance, e.g. 100x100 pixels. When you got that working you should add a global variable that controls the rotation of the camera. Use the Matrix class in **utils.pde** to represent the rotation matrix of the camera:

```
static Matrix3x3 R = new Matrix3x3();
```

Then make the left and right arrow keys rotate the camera around the y-axis. It might be convenient to add another global variable that stores the angle which the camera should be rotated around the y-axis:

```
static final float yaw = 0.05;
```

When the left and right arrow keys are pressed this angle should be updated and then the rotation matrix R should be updated to represent a rotation around the y-axis with this angle. If you do not remember how such a matrix looks you can have a look in your linear algebra book or at Wikipedia. If the camera is rotated by the matrix R then vectors representing the right (x-axis), down (y-axis) and forward (z-axis) directions can be retrieved from the elements of the matrix.

To model a rotating camera you need to use these directions both when you move the camera and when you cast rays. Implement this. As mentioned previously, you probably need to set a low width and height of the image for the algorithm to be fast enough for real-time motion. Since raytracing is slow it is typically not used for games and other interactive real-time applications. It is mainly used to render single images and special effects for movies. In lab 3 we will implement another algorithm, rasterization, which is faster and thus more suitable for interactive applications. After you got the camera motion working you can change back to a higher image resolution when doing the rest of the lab.

5 Illumination

You should now have a raytracer in which you can move the camera around. However, so far all points of a surface has exactly the same color. To add some more realism to the image we will add a light source. We will model an omni light. Such a light spreads light equally in all directions from a single point in space. The light can be defined by two global variables: its position and power for each color component:

```
static PVector lightPos = new PVector(0,  -0.5, -0.7);

static PVector lightColor = new Vector(1*14.f, 1*14.f, 1*14.f);
```

The color vector describes the power P, i.e. energy per time unit E/t of the emitted light for each color component. Each component thus has the physical unit W=J/s. The light will spread uniformly around the point light source, like an expanding sphere. Therefore, if a surface is further away from the light source it will receive less light per surface area. To derive a formula for the received light at a distance r from the light source we can think about a sphere with radius r centered at the light source. The emitted light will be spread uniformly over the whole surface area of the sphere which is:

$$A = 4\pi r^2 \qquad (25)$$

The power per area

Breaching any point on the sphere is therefore:

$$B = P/A = P/4\pi r^2 \qquad (26)$$

When working with illumination this physical quantity is **oft**en used. Since it describes power per area, it has the unit $W/m^2 = Js^{-1}\ m^{-2}$. Equation 26 describes the power per area reaching any surface point directly facing the light source. However, most of the time we will have surfaces that do not directly face the light source. Let $\hat{n}$ be a unit vector describing the normal pointing out from the surface and let $\hat{r}$ be a unit vector describing the direction from the surface point to the light source. Then if B is the power of light reaching a virtual surface area directly facing the light source at the surface point, then the power per real surface D will just be a fraction of this:

$$D = B \max(\hat{r}.\hat{n},0) = (P \max(\hat{r}.\hat{n},0))/4\pi r^2 \qquad (27)$$

We get the fraction between D and B by the projection of $\hat{r}$ on $\hat{n}$, i.e. a scalar product of two unit vectors. Since we do not allow negative light we take the max of the projected value and zero. If the angle between the surface normal and direction to the light source is larger than 90 degrees it does not receive any direct illumination. One way to think about this projection factor is to consider light as a stream of particles. If the particles hit the surface from an angle, instead of straight ahead, they will spread out more over the surface and less particles will hit a fixed area over a fixed time.

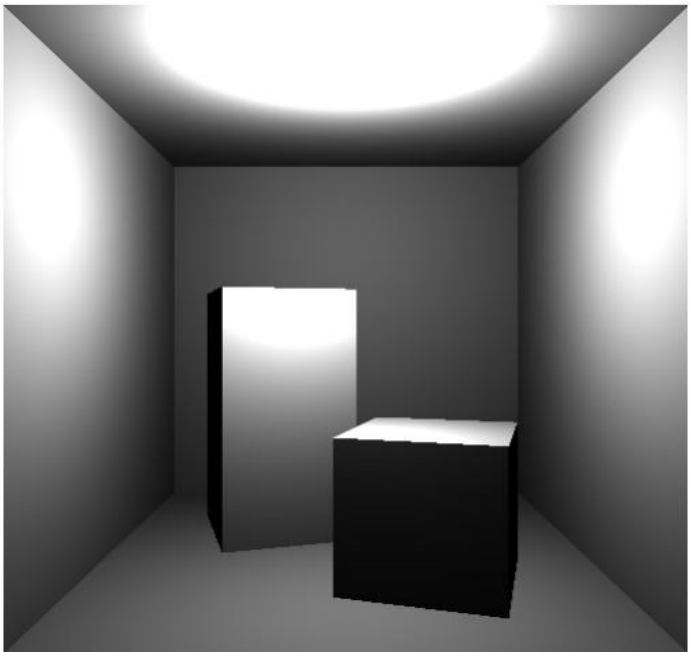

Figure 4: The incoming light.

If it is raining and you place a bucket outside it will fill more quickly if it is raining straight from above than if the wind is blowing and the rain comes from the side. Now we know how much light that reaches any surface point directly from the light source, i.e. without reflection via another surface. You should then implement the function:

```
PVector DirectLight(Intersection i){

	//your code here…

}
```

It takes an intersection, which gives us the position where we want to know the direct illumination, as well as the index to the triangle that should get illuminated, therefore we also know the normal of the surface. The function should then return the resulting direct illumination described by **Equation 27**.

Modify the Draw function such that it uses the function DirectLight to compute the color for every pixel, after you have computed the intersection it corresponds to. You should then get the result of Figure 4. Then you should also make it possible to move the light source, using the keys W and S to translate it forward and backward and A and D to translate it left and right and Q and E to translate it up and down. Do this in the **keyPressed()** function by writing something like:

```
case 'w':

	lightPos.z += delta;
```

```
break;
```

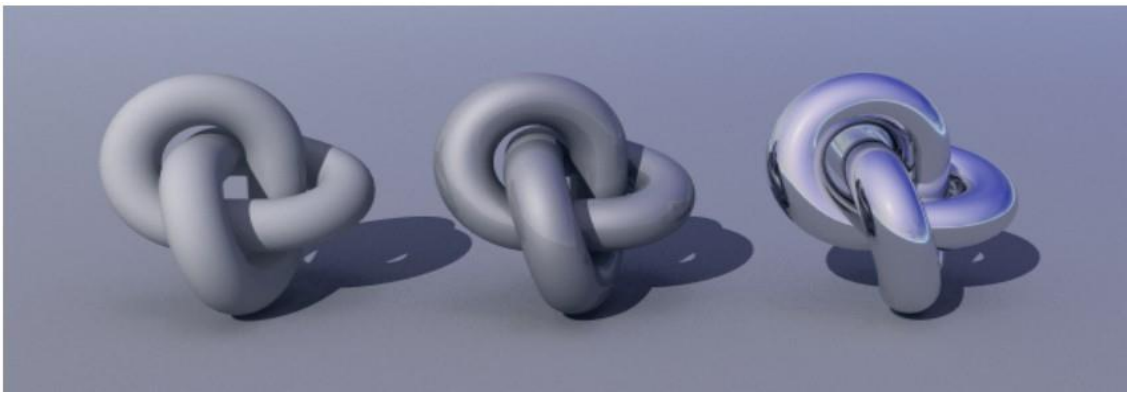

Figure 5: The left object is ideally diffuse. The right object is ideally specular. The middle object is somewhere in between.

However, this result does not correspond to how we would perceive the surface in the real world. It just tells how much light that reaches a surface point. What we actually see is the fraction of this that gets reflected in the direction of the ray. We thus need a model for this.

We will model all surfaces as ideally diffuse (see **Figure 5**). Such surfaces are simple to simulate because they reflect light equally in all directions. The viewing angle does not influence the perceived color of such a surface point. It will look the same from all directions, having no specular high lights. In that sense, it is the opposite of a mirror, which will look completely different depending on the viewing angle. A mirror has an ideally specular surface (see **Figure 5**).

Let $\rho = (\rho r, \rho g, \rho b)$ describe the fraction of the incoming light that gets reflected by the diffuse surface for each color component. Then the light that gets reflected is:

$$R = \rho * D = (\rho * P \max(\hat{r}.\hat{n},0)) / 4\pi r^2 \qquad (28)$$

where the operator **.** denotes element-wise multiplication between two vectors. If we use *Processing's* PVector for 3D vectors we can compute this element-wise multiplication by just writing:

```
PVector A = new PVector(1,2,3);
PVector B = new PVector(4,5,6);
PVector C = PVector.mult(A,B); // C will be (4,10,18)
```

Extend the Draw-function such that only the reflected light gets drawn. The color stored in each triangle is its ρ. This should give the result seen in **Figure 6**.

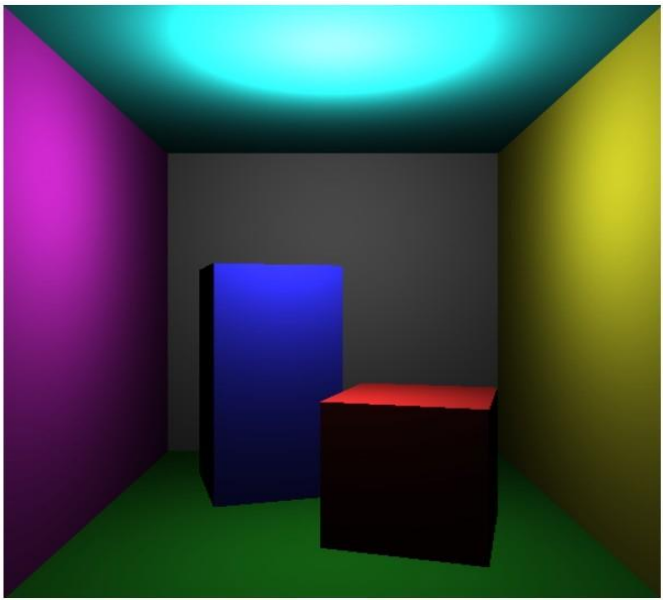

Figure 6: Direct illumination, without shadows.

5.1 Direct Shadows

So far a surface gets no direct illumination if it does not face the light source. However, we would also like to simulate shadows if another surface intersects the ray from the light source to the surface. To simulate this we can use the ClosestIntersection function that we use to cast rays. When illuminating a surface point we cast another ray from it to the light source. Then we check the distance to the closest intersecting surface. If that is closer than the light source the surface is in shadow and does not receive any direct illumination from the light source. Add this to your DirectLight function. If it works you should see shadows like in **Figure 7**.

5.2 Indirect Illumination

Just implementing the direct illumination will make some surface points appear pitch black. It only simulates one bounce of light from the light source via a surface point to the camera. In the real world light keeps reflecting multiple times and therefore reaches all surfaces at some point. At each reflection some of the light is absorbed by the surface, as specified by its ρ. This indirect illumination results in the smooth shadows that we see in the real world.

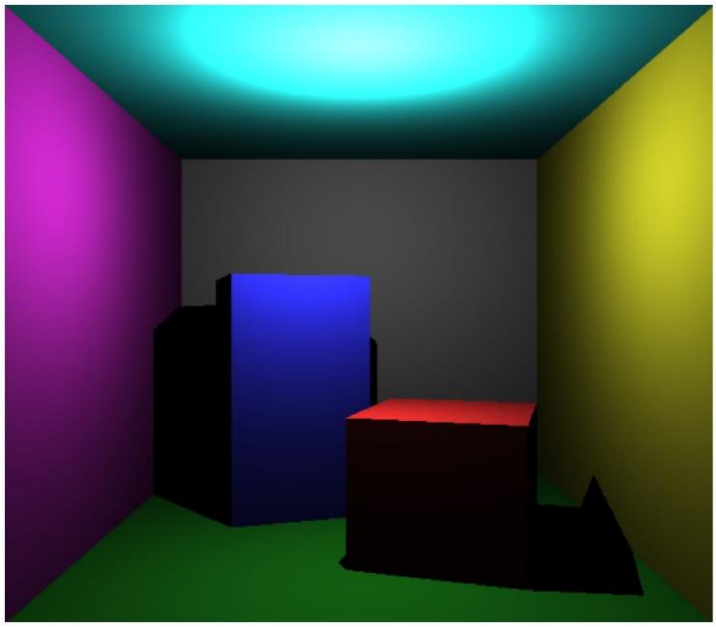

Figure 7: Direct illumination with shadows.

It is very computationally expensive to simulate the multiple bounces of light required for accurate computation of indirect illumination. For this lab it is not necessary. Instead, we will use a very simple approximation to model the indirect illumination. We will just assume that the indirect illumination is the same for every surface of the scene. Let N be the power per surface area of this constant indirect illumination. Then the total incident illumination of a surface point is:

$$T = D + N \qquad (29)$$

And the reflected light is:

$$R = \rho * T = \rho * (D+N) \qquad (30)$$

First create a new global variable to store the constant approximation of the indirect illumination:

```
static PVector indirectLight = new PVector(1* 0.5f, 1* 0.5f, 1* 0.5f);
```

Then implement this illumination model, **Equation 30**, in your Draw function. You should not alter the function DirectLight as it should only return the direct illumination which it already does. The result is an image were no surface is completely black, like in **Figure 8**.

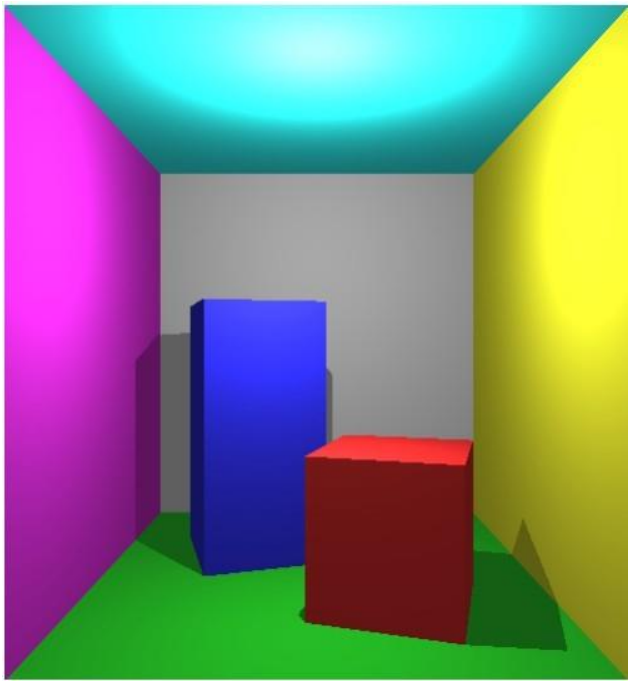

Figure 8: Direct illumination and constant approximation of indirect illumination

You have now written a raytracer, that handles direct illumination and shadows and approximates indirect illumination with a constant term. In the optional individual project, you have the opportunity to improve your raytracer. If you want you could then add things like:

- Soft edges (anti-aliasing) by sending multiple slightly perturbed rays for each pixel.
- Depth of field, by modeling the camera lens. Includes soft edges.
- Indirect illumination by multiple bounces of light (difficult).
- Textures. Can be used for color/reflectance and/or normal maps, and/or parallax mapping.
- Loading of general models.
- Storing the geometry in a hierarchical spatial structure for faster inter-section queries. This speeds up the rendering which is good for more complex/interesting scenes.
- Specular materials. For example you could simulate water with normal maps.
- Using fractals to automatically generate geometry and textures for mountains, clouds and water.